SPARK STRUCTURED STREAMING FUNDAMENTALS (WHAT & WHY)

1.1 DATA PROCESSING TYPES

Batch Processing

Data is processed in chunks.

Example:

- Daily report
- Nightly ETL job

```
Input → Stored → Process later → Output
```

Streaming Processing

Data is processed **as soon as it arrives**

```
Input → Process immediately → Output
```

Key Difference

Feature	Batch	Streaming
Data	Finite	Infinite
Latency	High	Low
Example	Daily report	Live dashboard

CORE IDEA

Streaming = **continuous flow of data**

1.2 PROBLEM WITH TRADITIONAL STREAMING

Earlier systems:

- Complex APIs
- Manual state handling
- Difficult failure recovery

1.3 SPARK APPROACH

Spark says:

Treat stream like a **table**

Concept

```
Stream = Unbounded Table
```

Example

Orders coming continuously:

```
order1
order2
```

```
order3
...
```

Table keeps growing forever

INTERNAL MODEL

Spark processes streaming using:

`Micro-batches`

Meaning

Instead of:

`1 record at a time`

Spark does:

`small batch → process → repeat`

EXERCISE

Q1:

Why can’t streaming data be stored completely?

Answer:

- Infinite data
- Memory limits
- Needs incremental processing

Q2:

Why is streaming needed?

Answer:

- Real-time analytics
- Immediate decisions
- Continuous pipelines

STRUCTURED STREAMING CORE

2.1 STREAMING DATAFRAME

Batch

```
df = spark.read.csv("file.csv")
```

Streaming

```
df = spark.readStream.csv("folder")
```

Key Difference

read	readStream

Batch	Streaming
Finite data	Infinite data

2.2 INPUT SOURCES

Spark supports:

- Files
- Kafka (most important)
- Socket

2.3 OUTPUT (SINKS)

Where results go:

- Console
- File
- Kafka
- Database

2.4 OUTPUT MODES

Append Mode

`Only new data added`

Complete Mode

Full result updated every time

Update Mode

Only changed rows updated

2.5 TRIGGERS

WHAT IS A TRIGGER IN SPARK STRUCTURED STREAMING?

A **trigger controls *when* Spark processes incoming data**

Think of it as:

“How often should Spark run a micro-batch job?”

CORE IDEA

Spark Structured Streaming works like:

Collect data → Process → Output → Repeat

Trigger decides **the timing of this loop**

1. DEFAULT TRIGGER (NO TRIGGER SPECIFIED)

Code:

```
df.writeStream \
  .format("console") \
  .start()
```

Behavior:

Run as fast as possible

What actually happens internally:

Batch 1 → process immediately
Batch 2 → process immediately
Batch 3 → process immediately

No waiting
Continuous execution

Important understanding:

- Spark keeps checking for new data
- As soon as data is available → process

Pros:

- Lowest latency
- Near real-time

Cons:

- More CPU usage
- Many small jobs

2. CUSTOM TRIGGER (PROCESSING TIME)

Code:

```
.trigger(processingTime="10 seconds")
```

Behavior:

Run micro-batch every 10 seconds

Timeline:

```
Time 0–10 sec → collect data → process
Time 10–20 sec → collect data → process
Time 20–30 sec → collect data → process
```

Important:

Even if data arrives earlier, Spark waits

Example:

You send data at:

Time 2 sec → data arrives

But:

Processing happens at 10 sec

Pros:

- Controlled execution
- Efficient batching
- Lower CPU usage

Cons:

- Higher latency
- Data not processed immediately

SIDE-BY-SIDE COMPARISON

Feature	Default Trigger	processingTime
Execution	Continuous	Fixed interval
Latency	Low	Higher
CPU usage	High	Lower
Control	No	Yes

3. WHEN TO USE WHAT?

Use DEFAULT when:

- Real-time dashboards
- Alerts
- Fraud detection

Use processingTime when:

- ETL pipelines
- Batch-like streaming
- Cost optimization

MINI LAB

Case 1 – Default

```
df.writeStream \
  .format("console") \
  .start()
```

Send Kafka data continuously
 Observe: output appears immediately

Case 2 – With Trigger

```
df.writeStream \
  .format("console") \
  .trigger(processingTime="10 seconds") \
  .start()
```

```
 Send Kafka data
  Observe: output comes every 10 seconds
```

2.6 STREAMING QUERY

```
df.writeStream.start()
```

This starts continuous processing

EXERCISES

Q1:

What is difference between append and complete mode?

Answer:

- Append → new data only
- Complete → entire table refreshed

Q2:

What happens if writeStream is not called?

Answer:

- Query does not start

HANDS-ON (FILE + KAFKA)

LAB 1 — FILE STREAMING

Step 1 — Create folder

```
mkdir -p /tmp/input
```

Step 2 — Spark Code

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.getOrCreate()

df = spark.readStream \
    .format("csv") \
    .option("header", "true") \
    .load("/tmp/input")

df.writeStream \
    .format("console") \
```

```
    .start() \
    .awaitTermination()
```

Step 3 — Add file

Create CSV file:

```
name,amount
A,100
B,200
```

Result

Spark reads file automatically

KAFKA STREAMING (MAIN LAB)

Step 1 — Kafka Source

```
df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "kafka:9092") \
    .option("subscribe", "orders") \
    .load()
```

Step 2 — Convert Data

```
df2 = df.selectExpr("CAST(value AS STRING)")
```

Step 3 — Output

```
df2.writeStream \
    .format("console") \
    .start() \
    .awaitTermination()
```

Step 4 — Send Kafka Data

```
order1,100
order2,200
```

Result

Spark prints instantly

EXERCISES

Q1:

Why CAST is required?

Answer:

- Kafka stores data as bytes

Q2:

What is subscribe?

Answer:

- Topic to read from

ADVANCED + REAL PIPELINE

4.1 TRANSFORMATIONS

Example: Split data

```
from pyspark.sql.functions import split

parsed = df2.select(
    split("value", ",")[0].alias("user"),
    split("value", ",")[1].alias("amount")
)
```

4.2 AGGREGATION (STATEFUL)

```
counts = parsed.groupBy("user").count()
```

4.3 OUTPUT MODE

```
counts.writeStream \
    .outputMode("complete") \
    .format("console") \
```

```
    .start() \
    .awaitTermination()
```

4.4 CHECKPOINTING

```
.option("checkpointLocation", "/tmp/chk")
```

Why needed?

- Recover from failure
- Maintain state

4.5 FINAL PIPELINE

```
df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "kafka:9092") \
    .option("subscribe", "orders") \
    .load()

df.selectExpr("CAST(value AS STRING)") \
.writeStream \
.format("parquet") \
.option("path", "/tmp/orders") \
.option("checkpointLocation", "/tmp/chk") \
.start()
```

FINAL EXERCISES

Q1:

What is micro-batch?

Answer:

- Streaming processed in small batches

Q2:

Why checkpointing?

Answer:

- Fault tolerance
- Recovery

Q3:

What happens if Spark crashes?

Answer:

- Recovers using checkpoint

1. EXECUTION MODEL

How Spark REALLY executes streaming

Even though we say "streaming", internally Spark does:

```
Micro-batch → Process → Micro-batch → Process → ...
```

Example timeline

```
Time 0–2 sec → batch1 → processed
Time 2–4 sec → batch2 → processed
Time 4–6 sec → batch3 → processed
```

Important Insight

Spark is NOT true event-by-event streaming
It is **micro-batch streaming**

Why micro-batch?

- Easier fault tolerance
- Easier scaling
- Reuses batch engine

Trigger Control

```
.trigger(processingTime="5 seconds")
```

👉 Controls batch frequency

Exercise

Q: If trigger = 10 seconds, what happens?

Answer:

- Data processed every 10 seconds
- Higher latency, fewer jobs

2. STATEFUL PROCESSING (CRITICAL)

◇ What is stateless?

```
df.select(...)
```

No memory of past data

◇ What is stateful?

```
groupBy().count()
```

Needs memory of previous data

Example

```
user1 → 3 orders
user2 → 5 orders
```

Spark must remember previous counts

Where is state stored?

On disk (checkpoint + state store)

Important concept

```
State grows over time
```

LAB — Stateful Aggregation

```
from pyspark.sql.functions import split

df2 = df.selectExpr("CAST(value AS STRING) as val")

parsed = df2.select(
    split("val", ",")[0].alias("user"),
    split("val", ",")[1].alias("amount")
)

counts = parsed.groupBy("user").count()

counts.writeStream \
    .outputMode("complete") \
    .format("console") \
    .start() \
    .awaitTermination()
```

- Counts keep increasing
- Spark maintains state internally

3. WINDOWING (REAL-WORLD USE CASE)

Why windowing?

Streaming is infinite → cannot group forever

So we group by **time windows**

◇ Example

```
Count orders every 10 seconds
```

LAB — Window Aggregation

```
from pyspark.sql.functions import window

windowed = df \
    .groupBy(window("timestamp", "10 seconds")) \
    .count()

windowed.writeStream \
    .outputMode("complete") \
    .format("console") \
    .start() \
    .awaitTermination()
```

Explanation

- timestamp column comes from Kafka
- window defines grouping interval

Types of Windows

Tumbling Window

```
[0–10], [10–20], [20–30]
```

Sliding Window

```
Window size = 10 sec
Slide = 5 sec
```

Overlapping windows

4. WATERMARK

Problem: Late Data

Example:

```
Event time: 10:00
Arrives at: 10:05
```

Without watermark

Spark waits forever

◇ With watermark

```
.withWatermark("timestamp", "5 minutes")
```

Accept late data up to 5 minutes

Behavior

Case	Result
Within watermark	processed
Beyond watermark	dropped

LAB — Watermark

```
df \
.withWatermark("timestamp", "1 minute") \
.groupBy(window("timestamp", "10 seconds")) \
.count()
```

- Late data handling
- Memory control

5. FAULT TOLERANCE

Checkpointing

```
.option("checkpointLocation", "/tmp/chk")
```

What is stored?

- Offsets
- State
- Progress

What happens on crash?

```
Restart → Resume from last checkpoint
```

Important

Without checkpoint → data loss possible

6. EXACTLY-ONCE PROCESSING

Spark guarantee

Exactly-once (with conditions)

Requirements:

1. Checkpointing
2. Idempotent sink

Kafka + Spark

- Kafka → source offsets tracked
- Spark → processes once

7. OUTPUT SINKS

Types

- Console (debug)
- File (data lake)
- Kafka (re-publish)
- DB (serving layer)

Example — Write to Kafka

```
df.writeStream \
.format("kafka") \
.option("kafka.bootstrap.servers", "kafka:9092") \
.option("topic", "output") \
.start()
```

8. PERFORMANCE CONSIDERATIONS

Input rate

- Kafka partitions
- Data volume

Processing speed

- Spark executors
- batch size

Backpressure

If input > processing:

```
Lag increases
```

Exercise

Q: What if Kafka sends data faster than Spark processes?

Answer:

- Lag builds up
- Delay increases

9. REAL-WORLD PIPELINE DESIGN

Example Architecture

```
App → Kafka → Spark → Data Lake → Dashboard
```

Steps

1. Ingest data (Kafka)
2. Process (Spark)
3. Store (Parquet)
4. Query (BI tools)

FINAL ADVANCED LAB

Goal

Count orders per user every 10 seconds

Code

```
from pyspark.sql.functions import split, window

df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "kafka:9092") \
    .option("subscribe", "orders") \
    .load()

df2 = df.selectExpr("CAST(value AS STRING) as val", "timestamp")

parsed = df2.select(
    split("val", ",")[0].alias("user"),
    split("val", ",")[1].alias("amount"),
    "timestamp"
)

result = parsed \
    .withWatermark("timestamp", "1 minute") \
    .groupBy(window("timestamp", "10 seconds"), "user") \
    .count()

result.writeStream \
    .outputMode("update") \
    .format("console") \
    .option("checkpointLocation", "/tmp/chk") \
```

```
.start() \
.awaitTermination()
```